

FULL BUILD BREAKDOWN

Every module, every integration, every page of the demo — what it is, how it works, and what it's built on.

<https://edagher92-coder.github.io/DOUGHBOSSV2/>

Staff / kitchen sign-in: `/staff.html` · Partner/licensing page: `/franchise.html`

Prepared by Elie D. for the Dough Boss ownership team · 1 July 2026

25

plugin classes / modules

13,339

lines of PHP (includes/ admin)

6

database tables

113

commits, incl. 16 dedicated hardening passes

This is a complete, dependency-free WordPress plugin — no Composer, no npm, no bundler — that gives Dough Boss its own commission-free ordering platform: storefront, kitchen board, payments, a POS bridge, a voucher engine, catering, and staff tooling, all built from scratch and reviewed repeatedly for security and accessibility. Everything below is grounded in the actual code, not a plan.

1 THE LIVE DEMO SITE — WHAT'S THERE TO CLICK THROUGH

The demo runs on GitHub Pages at the link on the cover. It's built with plain HTML/CSS/JS (no server needed to view it) and mirrors what the real WordPress plugin does. Four pages, each with its own job:

Page	What it shows
/ (index.html) Manoush, Pizza & Catering	The customer-facing storefront: browse the menu, build a pizza, see pricing. This is what a customer visiting the real site would use to order.
/staff.html Staff & Kitchen Sign-in	The front door for staff — a cinematic sign-in gate. Anything staff-only (the order board, the owner dashboard) sits behind this; there's no way to reach them without going through here first.
/backend.html Orders	The Live Order Board — what kitchen staff see after signing in. New orders, one-tap Accept → Preparing → Ready, the catering pipeline, and order history.
/owner.html Owner / Manager	The owner/manager dashboard — a read-only oversight view across all shops: live orders, catering pipeline, all-orders history, and daily figures.
/franchise.html Partner with Dough Boss	A licensing/wholesale partner page — the pitch and terms for anyone looking to open a licensed Dough Boss/Snow Boss outlet.

The demo is a front-end preview of the real plugin's UX — it uses placeholder/fictional order data so it's safe to show anyone, but the actual WordPress plugin behind it is real, working code (detailed below), not a mockup.

2 THE CORE PLATFORM — WHAT'S ACTUALLY BUILT

Module	What it does	Status
Storefront & pizza builder	Menu (custom post type), a build-your-own-pizza tool, and a cookie/token-based guest cart — no account required to order.	Live
Checkout & pricing	Pickup/delivery checkout. Every price is recalculated on the server from scratch on every request — the browser's numbers are never trusted, so nothing can be tampered with. AUD with GST built in correctly.	Live
Card payments (Stripe)	Full PaymentIntents flow: card entry, server-side confirmation, replay protection (a payment can't be reused on a second order). Switched off until live keys are entered.	Built · off
Live Order Board (kitchen)	Real-time kitchen screen. Orders appear instantly (see §3 — Mercure), with a 7-second automatic re-check as a safety net if the instant connection ever drops.	Live
Vouchers & discounts	Typo-resistant codes with a built-in check digit, QR-code claim/scan, and redemption that's provably impossible to use twice — even if two people scan the same code at the same instant. Includes the \$5/\$10 student campaign sharing one daily-capped pool.	Live
Catering	Enquiry → quote → deposit → balance → fulfilled pipeline, with its own database table and a quote engine that (like checkout) never trusts numbers from the browser.	Live
Multi-shop routing	Every order, voucher and shop setting is location-aware. One shop is configured today; more can be added through the admin without any new code.	Built · 1 shop configured
Staff Console app	A separate installable mini-app (Scan vouchers / manage vouchers / view the order board) that staff can use without ever logging into full WordPress admin.	Live
Notifications & printing	See §3 for the full breakdown — SMS, staff push alerts and kitchen receipt printing.	Built · off

3 INTEGRATIONS — THE POS BRIDGE AND EVERYTHING ELSE IT TALKS TO

Every integration below follows the same rule: **dependency-free** (no third-party SDK bundled — the plugin talks to each service directly over HTTP) and **fully dormant until explicitly configured**. If a setting isn't filled in, that feature doesn't run — the plugin behaves exactly as if it wasn't there.

POSPal (银豹) — the point-of-sale till bridge

This is the integration that connects the online voucher system to the physical till, so a voucher a customer claims online can be redeemed as a real discount when they pay in-store.

- **How it authenticates:** every request is signed — a timestamp plus an MD5 signature built from the shop's secret key and the request body. The secret key itself never appears in the request body, the URL, or any log — it's only ever used to compute that signature.
- **What it does:** when a customer claims a voucher, the plugin can grant it as a real member coupon on the shop's till (`grant_coupon()`); when it's redeemed, the plugin revokes it (`revoke_coupon()`) so it can't be used twice. Both of these are fully coded and connected end-to-end — this part of the build is finished.
- **Status at Revesby:** done. The \$5 and \$10 discount rules were created in the POSPal back office, entered into the plugin's settings, and verified live on 1 July 2026 — both rule IDs confirmed to match

real POSPal coupon rules. Recommended: one real end-to-end test (claim → grant → redeem → revoke) before relying on it for customers.

- **Current reach:** POSPal is connected at Revesby only; connecting additional shops means requesting POSPal credentials for each one and entering them the same way.

Stripe — card payments

PaymentIntents (not the older Charges API), created and verified entirely server-side. The secret key never leaves the server; the publishable key is the only thing sent to the browser. A completed payment is checked against the order amount and currency before the order is accepted, and the same payment can't be attached to two different orders.

Mercure — instant kitchen updates

Replaces the old "refresh every few seconds" kitchen board with real push updates: the moment an order is created or changes status, a tiny, privacy-safe signal (just "something changed, go refresh" — no customer details) is pushed to the board over Server-Sent Events. The board then re-fetches the real order data through the normal secure, logged-in route — the push message itself is never treated as trustworthy data. If Mercure isn't configured, the board simply falls back to checking every 7 seconds, so nothing ever breaks from this being off.

ClickSend — customer SMS

Texts a customer when their order is ready for pickup, and optionally when they claim a voucher. Authenticates with HTTP Basic auth (a username + API key, never logged). Phone numbers are auto-formatted for Australia. If SMS isn't configured, message sending is skipped silently — checkout and order flow are unaffected either way.

ntfy — staff push alerts

A near-free way to ping kitchen staff's phones the instant a new order comes in, separate from the order board itself. Carries no customer contact details — just a short "new order" heads-up.

Cloud receipt printing — CloudPRNT / Epson

Supports two printer protocols (Star CloudPRNT and Epson Server Direct Print). Unusually, there's no on-site print server required — the printer itself polls the plugin and pulls the next unprinted order as a ticket. Every poll must present a secret printer token or it's rejected. This is the kind of detail that makes "just add a receipt printer" simple later, since there's nothing extra to host.

4 UNDER THE HOOD — DATA, SECURITY & SCALE

Database

Six dedicated tables, all created and kept current automatically on plugin update:

- `doughboss_orders` & `order_items` — every order and its line items
- `doughboss_locations` — the shops
- `doughboss_catering_enquiries` — the catering pipeline
- `doughboss_vouchers` & `voucher_redemptions` — the voucher engine plus a permanent, unchangeable log of every redemption

Access & roles

Two dedicated capabilities and a purpose-built `doughboss_kitchen` WordPress role, so a kitchen tablet login can be handed to staff with access to only the order board and voucher scanning — nothing else in WordPress admin.

Security principles applied throughout

- Every price and total is recomputed on the server — the browser is never the source of truth for money.
- All database queries use parameterised statements; nothing is built by pasting values into SQL.
- All state-changing actions require a verified session token; public read-only data (menu, shop list) is intentionally open.
- Every secret (Stripe key, POSPal key, SMS key, printer token) is read from environment/settings, never hard-coded, never logged.

Review history

The codebase has been through **16 separate hardening passes** — dedicated review rounds covering security, WCAG accessibility (colour contrast, screen-reader labels, keyboard navigation), and everyday usability bugs — not a single pass, but repeated stress-testing against its own edge cases.

5 WHAT'S LEFT

The remaining work is short and specific — business inputs and setup decisions, not open engineering problems:

Item	What it needs
Additional shops	POSPal credentials + shop details for each new location beyond Revesby, which is now connected and verified (see §3).
Real menu & business details	Final items/prices/photos, each shop's address/hours/delivery area, and ABN for tax invoices.
Brand name confirmation	Lock in "Dough Boss" (vs. "Bite Pizzas" as a working name) before anything goes to print.
Switch-on decisions	Card payments, SMS, staff push and printing are all built — each just needs an account/keys and a go-ahead.
Hardware & hosting	A kitchen tablet + wifi per shop, and moving off any free/basic hosting plan before real orders go live.

Everything in this document is built and sitting in the codebase today. What turns it into a live, revenue-taking system is a short list of decisions and setup steps — not more development.

Prepared by Elie D. for internal use. Figures reflect the codebase as of 1 July 2026 (plugin v2.12.1). The demo site shows placeholder/fictional data for privacy; the plugin code it previews is real and described accurately above.